EEG Brain Speller

Brain-Computer Interface Project Report

Based on : BCI Competition III Dataset II

Project Overview:

The goal of this project was to build a working P300-based Brain-Computer Interface (BCI) speller from scratch. A BCI speller lets a person type characters using only their brain signals — no hand movement involved. The specific paradigm used here is the P300 Speller, one of the most well-studied and reliable BCI setups in the field.

The way it works is straightforward: a 6×6 grid of 36 characters is shown on screen. Rows and columns flash one at a time in a random order. Each time the row or column containing the character the user is focusing on lights up, the brain produces a distinctive electrical signal called the P300 component, which appears roughly 300 milliseconds after the flash. A trained classifier picks up on this signal and figures out which row and which column had the strongest P300 response — and their intersection gives the intended character.

The dataset used is the BCI Competition III Dataset II, which contains EEG recordings from two subjects (A and B). Each subject has a training set (85 characters) and a test set (100 characters), all recorded at 240 Hz across 64 EEG channels. The training data includes ground-truth labels for every flash so models can be trained; the test data only has a ground-truth string used at the end to measure how accurately the system decoded the characters.

Three classifiers were built and compared: LDA (Linear Discriminant Analysis), SVM (Support Vector Machine), and EEGNet (a compact convolutional neural network designed specifically for EEG). Each was evaluated at five repetition levels — 3, 5, 7, 10, and 15 — to understand how accuracy and speed trade off against each other.

Signal Processing Pipeline:

The system is organized into multiple modules, each responsible for a specific task. The role of each file in the pipeline is described below:

load_data.py

This is the first step in the pipeline. It loads the raw .mat files from the BCI Competition dataset using scipy's loadmat function. The function extracts four things from each file: the EEG signal itself (a 3D array of shape epochs × time samples × channels), the Flashing array which records when the screen was flashing, the StimulusCode array which tells you which row or column (1 through 12) was being flashed at each moment, and the StimulusType array which is only present in training files and tells you whether each flash was a P300 (1) or not (0). It also pulls out the target character string if available. Everything gets cast to the appropriate data type (float64 for the signal, int for the stimulus arrays) and a quick shape printout confirms the data loaded correctly.

preprocess.py

This file handles epoch extraction. The continuous EEG recording is split into small windows, one per flash event. Whenever the Flashing signal goes from 0 to 1 (a rising edge), that marks the start of a new flash. A 240-sample window (equivalent to 1 second at 240 Hz) is extracted from the signal at that point across all 64 channels. The stimulus code for that flash is also saved so we know which row or column it was. If the training labels are available, the P300/non-P300 label for that flash is saved too. Flashes that occur too close to the end of the recording are skipped to avoid reading out of bounds. The result for each subject is a list of epoch arrays (one per character), a corresponding list of labels, and a list of stimulus codes.

features.py

This file converts raw epochs into feature vectors that the classifiers can actually work with. It has two modes depending on which model is being used.

For LDA and SVM, it takes the portion of each epoch from sample 20 to 120 — which corresponds to roughly 83 to 500 milliseconds post-flash, the window where the P300 component lives. It then downsamples by a factor of 4 to get 25 timepoints per channel, and applies xDAWN spatial filtering using the pyriemann library. xDAWN is fitted on the training data and learns 6 spatial filters that maximise the signal-to-noise ratio of the P300 response across the 64 channels. The output is a 6×25 = 150-dimensional feature vector per epoch.

For EEGNet, the file instead takes samples 20 to 180 (downsampled by 2, giving 80 timepoints), and keeps the data in its 3D shape of channels × timepoints rather than flattening it. EEGNet needs the spatial structure intact because its convolutional layers learn from it directly. No xDAWN is applied in this mode.

model.py

This file just returns whichever classifier is requested by name. For LDA it returns a standard sklearn LinearDiscriminantAnalysis with default settings. For SVM it returns a LinearSVC wrapped in CalibratedClassifierCV — the wrapping is necessary because LinearSVC on its own does not output probabilities, and the decoding step needs probabilities. The reason a linear SVM was chosen over an RBF kernel SVM is purely practical: with around 15,000 training epochs, an RBF kernel would take an impractically long time to train because of its quadratic scaling with dataset size.

For EEGNet, the file builds a braindecode EEGClassifier around PyTorch's EEGNet architecture. It is configured with the correct number of channels and timepoints, trained for 5 epochs with a batch size of 64 using the Adam optimiser and cross-entropy loss.

decode.py

This file takes the P300 probabilities output by the classifier for each flash in a character trial and turns them into a character prediction. For each flash, the model outputs a probability that it was a P300. These probabilities are grouped by their stimulus code (1 through 12, where 1-6 are columns and 7-12 are rows). Within each group the probabilities are averaged together. The column with the highest average score and the row with the highest average score are identified, and those two indices are used to look up the character in the 6×6 grid.

The function also accepts an n_reps parameter which controls how many repetitions worth of flashes to use. At n_reps=3 only the first 36 flashes (3 repetitions × 12 codes) are used per character. At n_reps=15 all 180 flashes are used. This is how the speed-accuracy tradeoff is studied.

evaluate.py

This file contains all the measurement functions. The char_accuracy function compares the predicted character string against the ground-truth string and prints the percentage of characters that matched. The classification_metrics function computes precision, recall, and F1 on the epoch-level predictions — these measure how well the model distinguishes individual P300 flashes from non-P300 flashes. The stratified_kfold_cv function runs 5-fold cross-validation on the training data, keeping the class ratio consistent across folds. The plot_confusion_matrix function generates a heatmap of true vs predicted labels at the epoch level. Finally, calculate_itr computes the Information Transfer Rate in bits per minute using the standard BCI formula, which combines classification accuracy and trial duration into a single measure of communication speed.

speller_ui.py

This file implements the visual P300 speller interface using PsychoPy. It renders the 6×6 character grid on screen and simulates the flashing paradigm — rows and columns light up in a randomised order, pausing briefly between each flash. After all repetitions are done for a character, the decoded result is appended to the output string displayed at the top of the screen. The interface was built to demonstrate what the speller would look like in a real session. In the actual evaluation, the UI is optional and is only launched if SHOW_UI is set to True in main.py.

main.py

This is the entry point that runs everything in order. For each subject and the chosen model type, it loads the training data, extracts epochs, builds features, trains the classifier, then loads the test data, runs decoding at each repetition count, and prints accuracy and ITR. For LDA and SVM it also prints the epoch-level metrics and cross-validation results. At the end, if SHOW_UI is enabled, it launches the PsychoPy interface showing the decoded strings.

Results:

1. Epoch-Level Classification Metrics

These metrics measure how well each model distinguishes P300 from non-P300 at the level of individual flashes, before any averaging or decoding happens. They were computed on the training data. EEGNet is excluded here as braindecode does not expose sklearn-compatible predict() for this evaluation.

Subject / Model	Precision	Recall	F1 Score	5-Fold Accuracy
A — LDA	0.6553	0.1514	0.2459	0.8383
A — SVM	0.6762	0.1029	0.1786	0.8377
B — LDA	0.7332	0.4562	0.5625	0.8735
B — SVM	0.7598	0.4054	0.5287	0.8720

Table 1. Epoch-level precision, recall, F1, and 5-fold cross-validation accuracy for LDA and SVM on training data.

The low recall is expected — with roughly 11 non-P300 epochs for every 1 P300 epoch, the classifiers are conservative by nature and only predict P300 when they are confident. This is fine because the score-averaging decoding step handles the imbalance implicitly. Subject B's much higher recall and F1 reflects a stronger and more consistent P300 signal.

2. Character-Level Accuracy vs. Repetitions

This is the primary result — how many of the 100 test characters each model decoded correctly at each repetition count.

Model	3 reps	5 reps	7 reps	10 reps	15 reps
Subject A					
LDA	35%	54%	65%	76%	91%

Model	3 reps	5 reps	7 reps	10 reps	15 reps
SVM	39%	53%	68%	75%	92%
EEGNet	40%	46%	61%	76%	85%
Subject B					
LDA	70%	82%	85%	88%	90%
SVM	72%	83%	85%	86%	91%
EEGNet	64%	74%	86%	89%	**95%**

Table 2. Character-level decoding accuracy (%) on the 100-character test set. Bold = best result overall.

Subject B consistently outperforms Subject A at every repetition count across all models. The best single result was EEGNet on Subject B at 15 reps with 95% accuracy. LDA and SVM perform almost identically throughout — the xDAWN spatial filtering does most of the heavy lifting and the choice of linear classifier barely matters. EEGNet eventually surpasses the classical models on Subject B at higher rep counts but falls short on Subject A, likely because Subject A's weaker P300 signal requires more training data or training epochs than the 5 used here.

3. Information Transfer Rate (bits/min)

ITR combines accuracy and trial duration into a single communication speed metric. As repetitions increase, accuracy improves but each trial takes longer, so ITR can decrease even as accuracy goes up. Trial duration T = n_reps × 12 × 0.175 s.

Model	3 reps	5 reps	7 reps	10 reps	15 reps
Subject A					
LDA	8.59	10.37	9.96	8.98	8.14
SVM	10.25	10.07	10.71	8.79	8.30
EEGNet	10.68	8.03	9.00	8.98	7.22
Subject B					
LDA	26.19	20.38	15.47	11.50	7.98
SVM	**27.41**	20.80	15.47	11.05	8.14
EEGNet	22.67	17.20	15.79	11.73	8.81

Table 3. Information Transfer Rate (bits/min) at each repetition count. Bold = peak ITR overall.

For Subject A, peak ITR sits around 5–7 reps at roughly 10–11 bits/min across all models. At 15 reps accuracy is highest but the longer trial time pulls ITR down to around 8 bits/min. For Subject B, the peak is at 3 reps —

27.41 bits/min for SVM and 26.19 for LDA — because the P300 signal is strong enough that very few repetitions already give reliable decoding, and the short trial time gives a much higher communication rate. The overall best ITR was 27.41 bits/min from SVM on Subject B at 3 reps.

4. Summary of Key Findings

- Best accuracy: 95% — EEGNet, Subject B, 15 repetitions.
- Peak ITR: 27.41 bits/min — SVM, Subject B, 3 repetitions.
- LDA and SVM are effectively equal throughout; xDAWN spatial filtering is the dominant factor in performance, not the choice of linear classifier.
- EEGNet benefits from high-SNR subjects and more repetitions. With Subject B at 15 reps it outperforms classical models, but with Subject A's weaker signal and only 5 training epochs it does not catch up.
- Subject B's stronger P300 response allows high accuracy at far fewer repetitions, which is the key practical advantage — in a real BCI session this translates to a much faster communication rate.
- For practical deployment: Subject B-type users should run at 3–5 reps for maximum ITR; Subject A-type users need 7–10 reps to reach usable accuracy.